Reproducibility Report for League of Legends Data Mining Analysis

April 24, 2026

1 Objective

The objective of this analysis is to understand the key factors influencing match outcomes and player skill levels in League of Legends. Two classification problems were studied: predicting whether a player's team won the match, and predicting a player's ranked tier based on gameplay statistics.

2 Data Description

2.1 Dataset

The dataset used in this project is a League of Legends match dataset. The data contains gameplay-related variables such as kills, deaths, assists, gold earned, damage dealt, damage taken, vision-related statistics, and ranked tier information.

2.2 Classification Problems

I defined two classification problems here:

- **Binary classification:** Predict `win`, where 1 represents a win and 0 represents a loss.
- **Multi-class classification:** Predict `solo_tier`, which represents the player's rank tier.

2.3 Preprocessing Steps

I preprocesssed The dataset by removing invalid or corrupted entries from the `win` and `solo_tier` variables. The `win` variable was converted into a binary numeric format. Irrelevant identifier columns, such as match IDs and player IDs, were removed to avoid introducing noise. Besides, I selected a subset of meaningful gameplay-related numerical features for modeling. The dataset was then split into 90% training data and 10% validation data. Standardization was applied for KNN models due to their sensitivity to feature scales. In

Part C, I applied Principal Component Analysis (PCA) after scaling so that the number of features is reduced by half.

3 Methods

3.1 Models

Two classification models were used:

- **K-Nearest Neighbors (KNN)**
- **Random Forest**

KNN predicts a class label based on the majority class among the nearest neighboring data points. Random Forest is an ensemble model that combines multiple decision trees and uses majority voting for classification.

3.2 Hyperparameter Tuning

Five-fold cross-validation was used to tune hyperparameters on the training data.

- KNN: $k = [3, 5, 7, 9, 11]$
- Random Forest: `n_estimators` $= [50, 100, 150, 200, 300]$

The best hyperparameter value was selected based on the highest mean cross-validation accuracy.

3.3 Dimension Reduction

Principal Component Analysis (PCA) was used for dimension reduction. PCA transforms the original features into a smaller set of uncorrelated components. In this project, PCA was used to reduce the number of features by half.

4 Implementation

I implemented the program in Python using Jupyter Notebook. The main libraries used were:

- `pandas`
- `numpy`
- `scikit-learn`
- `matplotlib`

The general procedure was:

1. Load `binary_dataset.csv` and `multiclass_dataset.csv`.
2. Separate features and response variables.
3. Split each dataset into training and validation sets.
4. Perform 5-fold cross-validation on the training data.
5. Select the best hyperparameter value.
6. Train the final model on the full training set.
7. Evaluate performance on the validation set.
8. Repeat the same process after applying PCA.

5 Results

5.1 Binary Classification Results

Model	Best Hyperparameter	Training Accuracy	Validation Accuracy
KNN	$k = 11$	0.82	0.7912
Random Forest	$n = 200$	1.0	0.7926
KNN + PCA	$k = 11$	0.7962	0.772
Random Forest + PCA	$n = 150$	1.0	0.77

Table 1: Performance results for binary classification.

5.2 Multi-Class Classification Results

Model	Best Hyperparameter	Training Accuracy	Validation Accuracy
KNN	$k = 11$	0.3515	0.206
Random Forest	$n = 150$	1.0	0.257
KNN + PCA	$k = 11$	0.342	0.191
Random Forest + PCA	$n = 300$	1.0	0.2335

Table 2: Performance results for multi-class classification.

5.3 Cross-Validation Figures

Insert the cross-validation figures below.

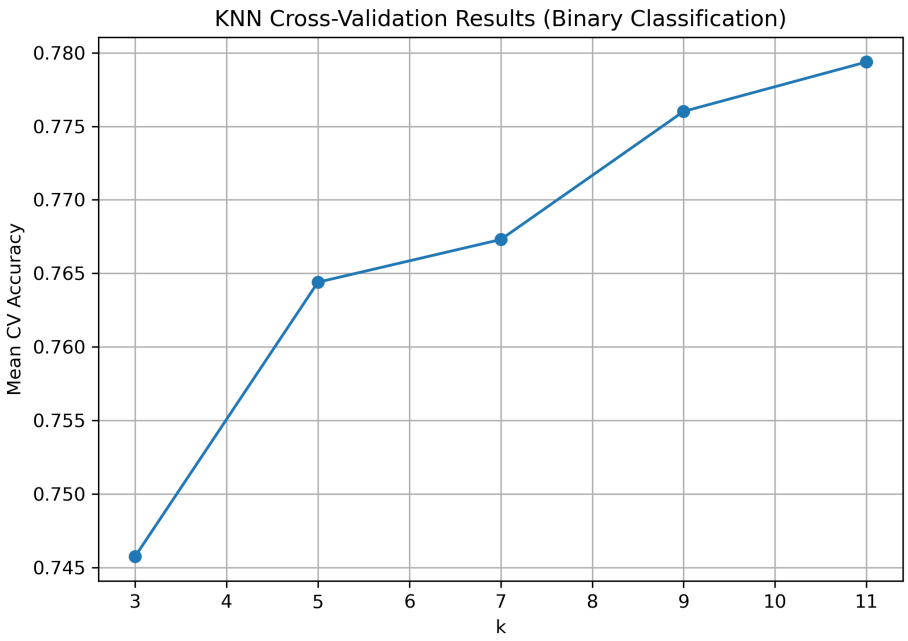

Figure 1: KNN cross-validation results for binary classification.

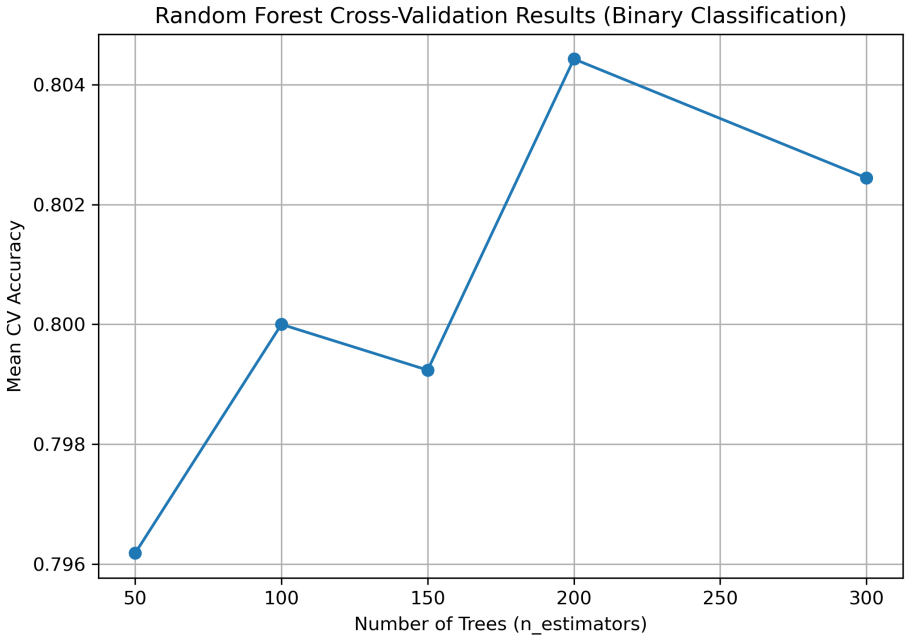

Figure 2: Random Forest cross-validation results for binary classification.

6 Discussion

6.1 Model Comparison

Random Forest consistently achieved slightly higher validation accuracy than KNN for both classification problems. For binary classification, Random Forest achieved a validation accuracy of approximately 0.793, compared to 0.791 for KNN. However, Random Forest showed clear signs of overfitting, with training accuracy reaching 1.0 while validation accuracy remained significantly lower. In contrast, KNN had lower training accuracy but better generalization behavior.

6.2 Binary vs Multi-Class Classification

The binary classification problem was significantly easier than the multi-class classification problem. The binary task achieved validation accuracies close to 0.79, while the multi-class task only achieved around 0.20–0.26. This indicates that predicting match outcomes is much more straightforward than predicting player rank tiers. The low multi-class accuracy suggests that rank tiers have overlapping feature distributions and are difficult to separate based on the available features.

6.3 Effect of PCA

PCA consistently reduced model performance for both KNN and Random Forest. For example, KNN accuracy decreased from approximately 0.791 to 0.772 in the binary task, and Random Forest accuracy decreased from 0.793 to 0.77. This suggests that PCA removed important information needed for classification. While PCA reduces dimensionality, it may discard discriminative features that are important for prediction.

6.4 Strengths and Limitations

A key strength of this analysis is the use of cross-validation and a separate validation set, which ensures reliable evaluation of model performance. However, a major limitation is the relatively low performance in the multi-class classification task. This suggests that the selected features may not fully capture the complexity of player skill levels. Additionally, Random Forest models exhibit overfitting, indicating that further regularization or tuning may be necessary.

7 Interpretation and Recommendations

Based on the results, Random Forest is recommended for the binary classification task because it achieved the highest validation accuracy. However, its tendency to overfit suggests that additional tuning or regularization may be necessary for practical deployment.

For the multi-class classification task, performance is relatively low for all models, indicating that the current feature set is insufficient to accurately distinguish between rank

tiers. Therefore, additional feature engineering or alternative modeling approaches should be considered before using the model in practice.

From a practical perspective, gameplay features such as kills, gold earned, and damage dealt appear to be useful predictors of match outcomes. Players can focus on improving these metrics to increase their chances of winning. Game developers can also use these insights to analyze game balance and player performance trends.

8 Future Steps

Future analysis could improve this project by:

- Testing additional models such as SVM or neural networks.
- Applying regularization techniques to reduce overfitting in Random Forest.
- Performing feature importance analysis to identify the most influential variables.
- Using confusion matrices to analyze misclassification patterns in the multi-class task.
- Addressing class imbalance to improve multi-class classification performance.
- Incorporating additional features such as champion roles, team composition, or objective control.